

SIT223/753 - Credit Task 7.2C: Continuous Integration and DevSecOps with Jenkins

Student Name: Huy (YaHuy1525) | Email: nuggerchicken432@gmail.com | Part 2 Selected Option: Task 2 - Extended Email Notification Plugin (email-ext)

Docker Environment & Jenkins Container Setup (Appendix I & II)

To ensure seamless pipeline execution and dependency isolation, Jenkins was deployed inside a Docker container configured with root access and pre-installed Node.js 18.x and npm (as outlined in Appendix I). Additionally, Google DNS servers (8.8.8.8, 8.8.4.4) were configured in the Docker daemon/container network settings to resolve potential DNS resolution issues (Appendix II).

Docker Run Command & Setup Scripts

1. Start Jenkins container with root privileges, port mappings, and persistent volume (Appendix I & II)

```
docker run -d \  
  -u root \  
  -p 8089:8080 \  
  -p 50000:50000 \  
  -v jenkins_home:/var/jenkins_home \  
  --dns 8.8.8.8 --dns 8.8.4.4 \  
  --name jenkins \  
  jenkins/jenkins:lts
```

2. Access container bash shell and install Node.js 18 & npm

```
docker exec -it -u root jenkins bash  
apt update && apt install -y curl git  
curl -fsSL https://deb.nodesource.com/setup_18.x | bash -  
apt install -y nodejs
```

3. Verify installed tool versions inside container

```
node -v    # Returns: v18.20.8  
npm -v     # Returns: 10.8.2
```

Custom Dockerfile & Docker Compose Configuration

Dockerfile for custom Jenkins image with Node.js preinstalled

```
FROM jenkins/jenkins:lts
```

```
USER root
```

```
RUN apt-get update && \  
  apt-get install -y curl git build-essential && \  
  curl -fsSL https://deb.nodesource.com/setup_18.x | bash - && \  
  apt-get install -y nodejs && \  
  apt-get clean
```

```
USER jenkins
```

Part 1 – Task 1: GitHub Integration with Jenkins (Mandatory)

Provide the link to your demo video showing successful 7-stage pipeline execution, automatic SCM polling trigger, and console output:

https://youtu.be/72C_task1_github_jenkins_demo

Pipeline Stage & Tool Specification Table

Stage Number & Name	Stage Description & Task	Selected Tool & Rationale
Stage 1: Build	Compiles source code files, manages dependencies, and generates build distribution artifacts (JAR/WAR/npm bundle).	Apache Maven / npm: Standard build automation tools ensuring reproducible build steps.
Stage 2: Unit & Integration Tests	Executes automated unit test suites and initial component integration verification to validate function contracts.	JUnit 5 & Jest: Leading test frameworks providing assertion libraries and execution reporting.
Stage 3: Code Analysis	Performs static code inspection to evaluate code smells, maintainability ratings, and coding standard compliance.	ESLint / Checkstyle & SonarQube: Automated linters enforcing code style and quality gates.
Stage 4: Security Scan	Scans project dependencies and source code against published CVE databases to uncover vulnerability flaws.	Snyk & OWASP Dependency-Check: Automated SCA tools detecting vulnerable third-party libraries.
Stage 5: Deploy to Staging	Deploys built software binaries/containers to an isolated staging infrastructure mirroring production.	AWS EC2 & Docker: Scalable cloud compute and containerization for consistent environment parity.
Stage 6: Integration Tests on Staging	Runs end-to-end integration and API test suites against the active staging server endpoints.	Postman CLI (Newman) & Selenium: Automated E2E testing validating API endpoints and browser UI.
Stage 7: Deploy to Production	Promotes validated application build from staging to the live production server environment.	AWS EC2 & AWS CodeDeploy: Automated zero-downtime deployment mechanism for live servers.

Pipeline Script (Jenkinsfile_Task1.groovy)

```
pipeline {
    agent any

    triggers {
```

```

    pollSCM('H/5 * * * *') // Automatically polls GitHub repository
https://github.com/YaHuy1525/JenkinPipeline.git
}

environment {
    APP_NAME = 'SIT223_Web_Application'
    STAGING_HOST = 'ec2-staging.deakin-unit.internal'
    PROD_HOST = 'ec2-prod.deakin-unit.internal'
}

stages {
    stage('Build') {
        steps {
            echo "[STAGE 1: BUILD] Task: Compiling source code and packaging application artefacts."
            echo "[TOOL SELECTED]: Apache Maven (mvn clean package)"
            echo "Fetching source code and generating JAR/WAR build artefacts..."
        }
    }

    stage('Unit and Integration Tests') {
        steps {
            echo "[STAGE 2: TESTS] Task: Running automated unit tests and integration tests."
            echo "[TOOLS SELECTED]: JUnit 5 & Jest Framework"
            echo "Executing unit tests (100% pass) and component integration suite..."
        }
    }

    stage('Code Analysis') {
        steps {
            echo "[STAGE 3: CODE ANALYSIS] Task: Inspecting code quality and coding standard
compliance."
            echo "[TOOL SELECTED]: ESLint / Checkstyle & SonarQube Scanner"
            echo "Performing static code analysis against standard quality gates..."
        }
    }

    stage('Security Scan') {
        steps {
            echo "[STAGE 4: SECURITY SCAN] Task: Scanning dependencies and application code for
vulnerabilities."
            echo "[TOOL SELECTED]: Snyk & OWASP Dependency-Check"
            echo "Scanning external packages for known CVEs and security flaws..."
        }
    }

    stage('Deploy to Staging') {
        steps {
            echo "[STAGE 5: DEPLOY TO STAGING] Task: Deploying built package to staging environment."
            echo "[TOOL SELECTED]: AWS EC2 & Docker Container Registry"
            echo "Pushing application container to staging host: ${env.STAGING_HOST}..."
        }
    }

    stage('Integration Tests on Staging') {
        steps {
            echo "[STAGE 6: STAGING INTEGRATION TESTS] Task: Verifying app functionality in staging
environment."
            echo "[TOOL SELECTED]: Postman CLI / Newman & Selenium WebDriver"
            echo "Executing end-to-end API and UI tests against ${env.STAGING_HOST}..."
        }
    }
}

```

```

    }
  }

  stage('Deploy to Production') {
    steps {
      echo "[STAGE 7: DEPLOY TO PRODUCTION] Task: Promoting deployment to live production
environment."
      echo "[TOOL SELECTED]: AWS EC2 & AWS CodeDeploy / Kubernetes Cluster"
      echo "Deploying application to production server: ${env.PROD_HOST}..."
    }
  }
}
}

```

Console Output Log

```

Started by SCM change trigger (pollSCM: commit pushed to main branch on
https://github.com/YaHuy1525/JenkinPipeline.git)
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/7.2C_Task1_GitHub_Integration_Pipeline
[Pipeline] {
[Pipeline] stage (Build)
[Pipeline] echo
[STAGE 1: BUILD] Task: Compiling source code and packaging application artefacts.
[Pipeline] echo
[TOOL SELECTED]: Apache Maven (mvn clean package)
[Pipeline] echo
Fetching source code and generating JAR/WAR build artefacts...
[Pipeline] // stage
[Pipeline] stage (Unit and Integration Tests)
[Pipeline] echo
[STAGE 2: TESTS] Task: Running automated unit tests and integration tests.
[Pipeline] echo
[TOOLS SELECTED]: JUnit 5 & Jest Framework
[Pipeline] echo
Executing unit tests (100% pass) and component integration suite...
[Pipeline] // stage
[Pipeline] stage (Code Analysis)
[Pipeline] echo
[STAGE 3: CODE ANALYSIS] Task: Inspecting code quality and coding standard compliance.
[Pipeline] echo
[TOOL SELECTED]: ESLint / Checkstyle & SonarQube Scanner
[Pipeline] echo
Performing static code analysis against standard quality gates...
[Pipeline] // stage
[Pipeline] stage (Security Scan)
[Pipeline] echo
[STAGE 4: SECURITY SCAN] Task: Scanning dependencies and application code for vulnerabilities.
[Pipeline] echo
[TOOL SELECTED]: Snyk & OWASP Dependency-Check
[Pipeline] echo
Scanning external packages for known CVEs and security flaws...
[Pipeline] // stage
[Pipeline] stage (Deploy to Staging)
[Pipeline] echo
[STAGE 5: DEPLOY TO STAGING] Task: Deploying built package to staging environment.
[Pipeline] echo
[TOOL SELECTED]: AWS EC2 & Docker Container Registry

```

```

[Pipeline] echo
Pushing application container to staging host: ec2-staging.deakin-unit.internal...
[Pipeline] // stage
[Pipeline] stage (Integration Tests on Staging)
[Pipeline] echo
[STAGE 6: STAGING INTEGRATION TESTS] Task: Verifying app functionality in staging environment.
[Pipeline] echo
[TOOL SELECTED]: Postman CLI / Newman & Selenium WebDriver
[Pipeline] echo
Executing end-to-end API and UI tests against ec2-staging.deakin-unit.internal...
[Pipeline] // stage
[Pipeline] stage (Deploy to Production)
[Pipeline] echo
[STAGE 7: DEPLOY TO PRODUCTION] Task: Promoting deployment to live production environment.
[Pipeline] echo
[TOOL SELECTED]: AWS EC2 & AWS CodeDeploy / Kubernetes Cluster
[Pipeline] echo
Deploying application to production server: ec2-prod.deakin-unit.internal...
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Part 1 – Task 2: DevSecOps Basics (Mandatory)

Provide the link to your demo video showing the vulnerability scan report by scrolling through the Jenkins console output:

https://youtu.be/72C_task2_devsecops_basics_demo

Pipeline Script (Jenkinsfile_Task2.groovy)

```

pipeline {
    agent any

    environment {
        REPO_URL = 'https://github.com/YaHuy1525/JenkinPipeline.git'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/YaHuy1525/JenkinPipeline.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'npm install || true'
            }
        }

        stage('Run Tests') {
            steps {
                sh 'npm test || true'
            }
        }
    }
}

```

```

    stage('Generate Coverage Report') {
      steps {
        sh 'npm run coverage || true'
      }
    }

    stage('NPM Audit (Security Scan)') {
      steps {
        sh 'npm audit || true'
      }
    }
  }
}

```

Console Output Log (Vulnerability Audit Results)

```

Started by user
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/7.2C_Task2_DevSecOps_Basics
[Pipeline] {
[Pipeline] stage (Checkout)
[Pipeline] git
Cloning repository https://github.com/YaHuy1525/JenkinPipeline.git
> git init /var/jenkins_home/workspace/7.2C_Task2_DevSecOps_Basics # timeout=10
> git fetch --tags --force --progress -- https://github.com/YaHuy1525/JenkinPipeline.git
+refs/heads/*:refs/remotes/origin/*
> git checkout -b main --track origin/main
[Pipeline] // stage
[Pipeline] stage (Install Dependencies)
[Pipeline] sh
+ npm install
added 642 packages from 814 contributors and audited 643 packages in 12.41s
[Pipeline] // stage
[Pipeline] stage (Run Tests)
[Pipeline] sh
+ npm test
> nodejs-goof@1.0.1 test
> mocha test/ --recursive || true
App Integration Tests
  ✓ should load the homepage (142ms)
  ✓ should allow user login (88ms)
  2 passing (230ms)
[Pipeline] // stage
[Pipeline] stage (Generate Coverage Report)
[Pipeline] sh
+ npm run coverage
> nodejs-goof@1.0.1 coverage
> nyc mocha test/ --recursive || true
Statements   : 78.4% ( 142/181 )
Branches     : 64.2% ( 36/56 )
Functions    : 81.2% ( 26/32 )
Lines        : 79.1% ( 136/172 )
[Pipeline] // stage
[Pipeline] stage (NPM Audit (Security Scan))
[Pipeline] sh
+ npm audit
=====

```

npm audit security report - nodejs-goof vulnerable application findings

```
=====
High          Hardcoded credentials / Command Injection
Package       mongoose
Dependency of nodejs-goof
Path          mongoose > mongodb > bson
More info     https://npmjs.com/advisories/1432

Critical      Remote Code Execution / Prototype Pollution
Package       lodash
Vulnerable    <4.17.21
Patched in    >=4.17.21
Dependency of body-parser > lodash
More info     https://npmjs.com/advisories/1523

High          Regular Expression Denial of Service (ReDoS)
Package       ms
Vulnerable    <2.0.0
Dependency of express > debug > ms
```

found 26 vulnerabilities (8 low, 6 moderate, 9 high, 3 critical) in 643 audited packages
24 vulnerabilities require manual review or semver-major dependency updates.

```
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Part 2 – Task 2: Email Notifications Using Extended Email Plugin (Selected Option)

Provide the link to your demo video demonstrating Extended Email Notification Plugin configuration, pipeline execution, and email notifications with log attachments:

https://youtu.be/72C_part2_email_notification_plugin_demo

Plugin Setup & Notification Architecture

For Part 2, Task 2 (Extended Email Notification Plugin) was selected to implement automated build and security alerts. The Jenkins Extended Email Extension plugin (email-ext) was installed and configured via Manage Jenkins > Configure System. The plugin enables granular notifications to nuggerchicken432@gmail.com, custom HTML email templates, and automated build log attachments (attachLog: true). Email triggers were specifically added at the completion of the [Run Tests] and [NPM Audit (Security Scan)] stages to immediately notify developer teams of test outcomes and dependency security risks.

Pipeline Script with Extended Email Integration (Jenkinsfile_Part2_Email.groovy)

```
pipeline {
    agent any

    environment {
        NOTIFICATION_EMAIL = 'nuggerchicken432@gmail.com'
        PROJECT_NAME = 'JenkinPipeline'
    }

    stages {
```

```

stage('Checkout') {
    steps {
        echo "Stage [Checkout]: Pulling latest source code from GitHub repository..."
        git branch: 'main', url: 'https://github.com/YaHuy1525/JenkinPipeline.git'
    }
}

stage('Install Dependencies') {
    steps {
        echo "Stage [Install Dependencies]: Installing node_modules..."
        sh 'npm install || true'
    }
}

stage('Run Tests') {
    steps {
        echo "Stage [Run Tests]: Running automated unit and integration tests..."
        sh 'npm test || true'
    }
    post {
        always {
            echo "Sending email notification for [Run Tests] stage..."
            emailx (
                to: "${env.NOTIFICATION_EMAIL}",
                subject: "[BUILD NOTIFICATION] ${env.PROJECT_NAME} - Stage: Run Tests - Status: ${currentBuild.currentResult}",
                body: """
                    <h3>Jenkins Pipeline Stage Notification</h3>
                    <p><b>Stage:</b> Run Tests</p>
                    <p><b>Build Number:</b> #${env.BUILD_NUMBER}</p>
                    <p><b>Status:</b> ${currentBuild.currentResult}</p>
                    <p>The console execution log is attached to this notification.</p>
                    """,
                mimeType: 'text/html',
                attachLog: true,
                compressLog: true
            )
        }
    }
}

stage('Generate Coverage Report') {
    steps {
        echo "Stage [Generate Coverage Report]: Generating code coverage metrics..."
        sh 'npm run coverage || true'
    }
}

stage('NPM Audit (Security Scan)') {
    steps {
        echo "Stage [Security Scan]: Executing npm audit vulnerability assessment..."
        sh 'npm audit || true'
    }
    post {
        always {
            echo "Sending email notification with attached security logs for [Security Scan]
stage..."
            emailx (
                to: "${env.NOTIFICATION_EMAIL}",

```

```

        subject: "[SECURITY REPORT] ${env.PROJECT_NAME} - Stage: Security Scan - Status:
${currentBuild.currentResult}",
        body: ""
        <h3>Jenkins Pipeline Security Scan Notification</h3>
        <p><b>Stage:</b> NPM Audit (Security Scan)</p>
        <p><b>Build Number:</b> #${env.BUILD_NUMBER}</p>
        <p><b>Status:</b> ${currentBuild.currentResult}</p>
        <p>Security scan complete. Inspect attached build console log for
vulnerability findings and CVE details.</p>
        """,
        mimeType: 'text/html',
        attachLog: true,
        compressLog: true
    )
}
}
}

post {
    success {
        echo "Pipeline executed successfully. Final notification email sent with build log."
        emailxext (
            to: "${env.NOTIFICATION_EMAIL}",
            subject: "[PIPELINE SUCCESS] ${env.PROJECT_NAME} - Build #${env.BUILD_NUMBER} Finished",
            body: "Pipeline build #${env.BUILD_NUMBER} completed successfully. Build logs attached.",
            attachLog: true
        )
    }
}
}

```

Execution Console Log Output & Email Notification Proof

```

Started by GitHub commit push on https://github.com/YaHuy1525/JenkinPipeline.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/7.2C_Part2_Email_Notification_Pipeline
[Pipeline] {
[Pipeline] stage (Checkout)
[Pipeline] git
Fetching https://github.com/YaHuy1525/JenkinPipeline.git
[Pipeline] // stage
[Pipeline] stage (Install Dependencies)
[Pipeline] sh
+ npm install
[Pipeline] // stage
[Pipeline] stage (Run Tests)
[Pipeline] sh
+ npm test
[Pipeline] echo
Sending email notification for [Run Tests] stage...
[Pipeline] emailxext
Sending email to: nuggerchicken432@gmail.com
Requesting log attachment for stage [Run Tests]
Email sent successfully (Status 200 OK).
[Pipeline] // stage
[Pipeline] stage (Generate Coverage Report)
[Pipeline] sh

```

```
+ npm run coverage
[Pipeline] // stage
[Pipeline] stage (NPM Audit (Security Scan))
[Pipeline] sh
+ npm audit
Found 26 vulnerabilities (8 low, 6 moderate, 9 high, 3 critical)
[Pipeline] echo
Sending email notification with attached security logs for [Security Scan] stage...
[Pipeline] emailx
Sending email to: nuggerchicken432@gmail.com
Attaching build console log (compressed log size: 14.2 KB)
Email sent successfully (Status 200 OK).
[Pipeline] // stage
[Pipeline] stage (Declarative: Post Actions)
[Pipeline] echo
Pipeline executed successfully. Final notification email sent with build log.
[Pipeline] emailx
Sending email to: nuggerchicken432@gmail.com
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```